

Jogo da Velha com Python, POO e KivyMD

Apresentacao do Trabalho - Programacao Orientada a Objetos

1. Visao Geral do Projeto

Este projeto foi desenvolvido para a disciplina de Programacao Orientada a Objetos, com o objetivo de aplicar, na pratica, os principais conceitos vistos em aula, como classes, objetos, heranca, encapsulamento, polimorfismo, composicao e separacao de responsabilidades. Para isso, foi criado um jogo de tabuleiro utilizando Python.

O jogo escolhido foi o Jogo da Velha, pois permite trabalhar de forma equilibrada com regras, turnos, jogadores, tabuleiro, condicoes de vitoria e empate, sem tornar o projeto excessivamente grande ou complexo.

2. Desenvolvimento e Desafios

Um dos principais desafios do projeto foi separar corretamente a logica do jogo da parte visual. No inicio, parecia mais simples concentrar tudo em um unico lugar, mas essa abordagem deixaria o codigo dificil de manter. Por isso, optou-se pelo padrao MVC (Model-View-Controller), dividindo o sistema em Model, View e Controller.

Outro desafio foi aprender a trabalhar com o KivyMD e com arquivos no formato .kv, recursos introduzidos recentemente. Foi necessario entender como esses arquivos se conectam as classes Python, como funcionam os identificadores (ids), os botoes, as telas e o ScreenManager. A construcao da interface exige uma forma diferente de pensar, principalmente porque a tela precisa ser atualizada apos cada jogada.

Para auxiliar em problemas repetitivos e em duvidas durante o desenvolvimento, foi utilizada a IA Claude, que contribuiu com organizacao de codigo, correcao de erros simples, ideias para estruturar melhor as classes e pequenos ajustes na interface. Mesmo com esse apoio, o foco do projeto foi estudar o codigo e compreender o funcionamento de cada parte dentro da arquitetura orientada a objetos.

3. Como Executar o Projeto

Na raiz do projeto, instale as dependencias com o comando 'pip install -r requirements.txt'. Em seguida, execute a interface grafica com 'py main.py'. Tambem existe uma versao antiga em terminal, que pode ser executada com 'py -m src.main'. Para rodar os testes automatizados, utilize 'py -m pytest -v'.

4. Estrutura do Projeto

O projeto e organizado em pastas que separam claramente as responsabilidades: o arquivo main.py inicia a aplicacao; a pasta app/ contem os modulos models, views, controllers e components, referentes a interface grafica em KivyMD; a pasta assets/

guarda recursos visuais; a pasta src/ contem os modulos core e jogos, com as classes mais ligadas a Programacao Orientada a Objetos; e a pasta tests/ reúne os testes automatizados do projeto.

5. Arquitetura MVC

O projeto utiliza o padrao MVC para manter o codigo organizado e de facil manutencao:

- **Model:** localizado em `app/models/game_model.py`, representa os dados e a partida.
- **View:** localizada em `app/views/`, cuida das telas em KivyMD e dos arquivos `.kv`.
- **Controller:** localizado em `app/controllers/game_controller.py`, faz a ligacao entre a interface e a logica do jogo.
- **Core do jogo:** localizado em `src/core` e `src/jogos`, onde estao as classes mais ligadas a Programacao Orientada a Objetos.

A View nao acessa diretamente os atributos internos do jogo. Ela chama o Controller, e o Controller chama metodos publicos do Model. Essa estrutura evita misturar a interface grafica com as regras de negocio.

6. Telas Implementadas

- **Menu Principal:** apresenta o nome do jogo e as opcoes principais.
- **Configuracao de Partida:** permite informar o nome dos jogadores.
- **Tabuleiro:** exhibe o jogo, o jogador da vez, o placar e as jogadas.
- **Resultado:** mostra o vencedor ou empate, estatisticas e opcao de nova partida.

7. Componentes do KivyMD Utilizados

Foram utilizados diversos componentes do KivyMD, entre eles: `MDDApp`, `MDScreenManager`, `MDDTopAppBar`, `MDCard`, `MDRaisedButton`, `MDDialog`, `MDDLabel`, `MDDIcon` e `MDDGridLayout`, alem de elementos padrao do Kivy para composicao das telas.

8. Principais Conceitos de POO Aplicados

- **Abstracao:** classes como `JogoTabuleiro` representam uma ideia geral de jogo.
- **Heranca:** a classe `JogoDaVelha` herda de `JogoTabuleiro`.
- **Encapsulamento:** os atributos internos sao protegidos e acessados por metodos publicos.
- **Polimorfismo:** cada jogo pode implementar suas regras de forma propria.
- **Composicao:** o jogo e composto por tabuleiro, jogadores, pecas, regras e historico.

9. Conclusao

Este trabalho permitiu compreender, na pratica, como a Programacao Orientada a Objetos pode tornar um projeto mais organizado, modular e facil de manter. Tambem evidenciou que a construcao de uma interface grafica com KivyMD e arquivos `.kv` exige cuidado, ja que a tela precisa estar bem separada da logica do jogo. Apesar das dificuldades encontradas ao longo do desenvolvimento, o projeto resultou em uma aplicacao funcional, capaz de iniciar uma partida, permitir as jogadas, detectar o vencedor ou o empate e reiniciar o jogo.